

Informe Tarea 1: Procesamiento Masivo de Datos

Vicente Jiménez Marco Pérez

16 de septiembre de 2026

Resumen

En este informe se presentan las soluciones de las dos partes de la tarea. En la primera se emplean comandos de GNU/Linux y AWK para procesar los archivos `worldcitiespop.csv.gz` y `matrix.txt`, incluyendo agregaciones por país, ordenamientos, filtros y estadísticas por filas y columnas. En la segunda se describe el diseño, la implementación y la evaluación experimental de un sistema de mapeo genómico probabilístico sin verificación de alineamiento completo, comparando tres variantes: AWK interpretado, C secuencial y C concurrente con POSIX Threads. El modelo se basa en la indexación de k -mers muestreados a paso L mediante una tabla hash directa y votación por coincidencias. Se evaluaron datos de *Escherichia coli* y un subconjunto reproducible de *Homo sapiens*. Las variantes compiladas en C entregaron resultados idénticos y la versión paralela alcanzó una aceleración máxima de $1,57\times$ con 8 hilos respecto de C secuencial.

Nota sobre reproducibilidad y replicación:

Todos los códigos fuente, scripts de prueba, datasets y herramientas de automatización están estructurados para su ejecución determinista en el repositorio del proyecto. Para compilar los binarios optimizados, descargar y preparar las muestras de datos oficiales (*E. coli* y *Homo sapiens*), y replicar íntegramente los experimentos, tablas y tiempos expuestos en este informe, por favor siga las instrucciones detalladas en el archivo `README.md` ejecutando los objetivos `make data` y `make test`.

1 Parte 1: Procesamiento de datos con comandos Linux

La primera parte se resolvió mediante el script `run_problema1.sh`. El archivo define una función para cada inciso, desde `a` hasta `j`, y permite ejecutar una solución particular o todas de forma consecutiva. Esta organización evita repetir comandos manualmente y facilita la reproducción de los resultados.

1.1 Criterios generales de implementación

El archivo `worldcitiespop.csv.gz` se procesa sin descomprimirlo de manera permanente. El comando `gzip -dc` escribe su contenido descomprimido en la salida estándar y la tubería `|` lo entrega directamente a AWK. En las consultas sobre el CSV se utiliza `-F','`, pues la coma es el separador de campos. Además, se descarta la cabecera mediante la condición `NR > 1`.

En este conjunto de datos se emplean principalmente las siguientes columnas:

- `$1`: código del país;
- `$3`: nombre de la ciudad con acentos;
- `$5`: población informada.

Para ejecutar todo el problema se utiliza:

```
chmod +x run_problema1.sh
./run_problema1.sh all
```

También puede ejecutarse un solo inciso, por ejemplo `./run_problema1.sh d`. En los incisos a), c) y f), el script incluye `head` únicamente para mostrar una vista previa en pantalla. Para guardar la salida completa se debe retirar ese último filtro o redirigir la versión completa a un archivo.

1.2 Consultas sobre `worldcitiespop.csv.gz`

a) Promedio de habitantes por ciudad para cada país

Se acumula la población de las ciudades con valor conocido y positivo, y simultáneamente se cuenta cuántas ciudades válidas posee cada país. El promedio se calcula como

$$\overline{P}_c = \frac{\sum_{i \in I_c} P_i}{|I_c|},$$

donde I_c contiene las ciudades del país c cuya población está informada y es mayor que cero.

```
gzip -dc worldcitiespop.csv.gz | awk -F',' '
NR > 1 && $5 != "" && $5 > 0 {
    sum[$1] += $5
    count[$1]++
}
END {
    for (c in sum)
        printf "%s\t%.2f\n", c, sum[c] / count[c]
}' | sort -k1,1
```

La decisión de excluir campos vacíos y valores cero evita interpretar la ausencia de información como una ciudad de población nula. Por tanto, el denominador es el número de ciudades con población válida y no el total de registros del país.

b) Diez ciudades con mayor población

AWK conserva únicamente los registros cuya población es numérica y emite primero dicha población, seguida de la ciudad y el país. Colocar la población como primera columna permite aplicar directamente un ordenamiento numérico decreciente con `sort -k1,1nr`; finalmente, `head -n 10` selecciona las diez primeras filas.

```
gzip -dc worldcitiespop.csv.gz | awk -F',' '
NR > 1 && $5 ~ /^[0-9]+$/ { print $5, $3, $1 }
' | sort -k1,1nr | head -n 10
```

c) Porcentaje de ciudades sin población por país

Para cada país se cuenta el total de registros y, por separado, los que tienen el campo de población vacío o igual a cero. El porcentaje reportado corresponde a

$$q_c = 100 \frac{N_c^{\text{sin poblacion}}}{N_c^{\text{total}}}.$$

```
gzip -dc worldcitiespop.csv.gz | awk -F',' '
NR > 1 {
    total[$1]++
    if ($5 == "" || $5 == 0) no_pop[$1]++
}
```

```

}
END {
    for (c in total)
        printf "%s\t%.2f%%\t(%d/%d)\n", c,
            100 * no_pop[c] / total[c], no_pop[c], total[c]
}' | sort -k1,1

```

Además del porcentaje, se imprimen el numerador y el denominador para que el resultado pueda verificarse con facilidad.

d) Población total y ciudad más poblada de América del Sur

Se crea en BEGIN un conjunto asociativo con los códigos de los países y territorios sudamericanos. Luego se realiza una sola pasada sobre los datos: cada población válida se agrega al total y, cuando supera el máximo observado, se actualizan la ciudad, el país y su población.

```

BEGIN {
    split("ar bo br cl co ec fk gf gy pe py sr uy ve", sa, " ")
    for (i in sa) is_sa[sa[i]] = 1
    max_pop = -1
}
NR > 1 && ($1 in is_sa) && $5 ~ /^[0-9]+$/ {
    total_pop += $5
    if ($5 > max_pop) {
        max_pop = $5
        top_city = $3
        top_country = $1
    }
}

```

Se incluyeron `gf` y `fk` por su ubicación geográfica. El total obtenido debe interpretarse como la suma de las poblaciones disponibles en el archivo, no como un censo exhaustivo: los registros sin población no aportan al acumulado.

1.3 Consultas sobre `matrix.txt`

En los incisos siguientes, AWK separa automáticamente los campos por espacios en blanco. Cada registro corresponde a una fila y `NF` indica su número de columnas.

e) Mínimo y máximo de cada columna

Los arreglos `min[i]` y `max[i]` almacenan los extremos de la columna i . La primera fila inicializa ambos valores; las filas posteriores solo los actualizan cuando aparece un elemento menor o mayor. Esta inicialización es más segura que comenzar en cero, pues la matriz podría contener únicamente valores negativos o únicamente valores positivos.

```

awk '{
    for (i = 1; i <= NF; i++) {
        if (NR == 1 || $i > max[i]) max[i] = $i
        if (NR == 1 || $i < min[i]) min[i] = $i
    }
}
END {
    for (i = 1; i <= length(max); i++)
        printf "Col %d: Min = %d, Max = %d\n", i, min[i], max[i]
}' matrix.txt

```

El uso de `length(max)` supone GNU AWK, coherente con el entorno GNU/Linux solicitado. Para una implementación estrictamente portable podría guardarse explícitamente el máximo valor observado de `NF`.

f) Inversión del orden de las columnas

Se recorre cada fila desde NF hasta 1. La expresión condicional del `printf` inserta el separador de campos entre elementos y un salto de línea después del último.

```
awk '{
    for (i = NF; i >= 1; i--)
        printf "%s%s", $i, (i == 1 ? RS : FS)
}' matrix.txt
```

En el script se añade `head -n 3` para visualizar tres filas; la transformación completa corresponde al comando anterior sin ese filtro.

g) Filas con promedio mayor que el promedio general

Se requieren dos pasadas porque el promedio global debe conocerse antes de decidir qué filas imprimir. En la primera lectura, identificada por `NR == FNR`, se suman todos los elementos y se cuenta cuántos existen. En la segunda se calcula el promedio de cada fila y se compara con el global:

$$\bar{x} = \frac{1}{N} \sum_{r,j} x_{rj}, \quad \bar{x}_r = \frac{1}{n_r} \sum_j x_{rj}.$$

```
awk 'NR == FNR {
    for (i = 1; i <= NF; i++) {
        total_sum += $i
        total_cnt++
    }
    next
}
FNR == 1 { global_avg = total_sum / total_cnt }
{
    row_sum = 0
    for (i = 1; i <= NF; i++) row_sum += $i
    if (row_sum / NF > global_avg)
        print "Fila " FNR " (Prom: " row_sum/NF \
            " > Global: " global_avg ")"
}' matrix.txt matrix.txt
```

Al entregar dos veces `matrix.txt` como argumento, AWK lo lee dos veces dentro del mismo proceso. La variable `FNR` vuelve a 1 al comenzar el segundo archivo, mientras `NR` conserva el conteo acumulado.

h) Cantidad de números impares

Cada elemento se clasifica usando el resto módulo 2. Si `$i % 2 != 0`, se incrementa el contador de impares.

```
awk '{
    for (i = 1; i <= NF; i++)
        if ($i % 2 != 0) odds++
}
END { print "Total impares:", odds }' matrix.txt
```

i) Divisibilidad por 4, por 9 y simultánea

Las condiciones se contabilizan por separado para no perder los números divisibles por ambos. La intersección equivale a los múltiplos de $\text{mcm}(4, 9) = 36$, pero se comprueba directamente con la conjunción lógica.

```
awk '{
    for (i = 1; i <= NF; i++) {
        if ($i % 4 == 0) c4++
        if ($i % 9 == 0) c9++
        if ($i % 4 == 0 && $i % 9 == 0) c_both++
    }
}
END { printf "Por 4: %d; por 9: %d; por ambos: %d\n", \
        c4, c9, c_both }' matrix.txt
```

j) Fila con mayor suma acumulada

Para cada fila se reinicia `sum`, se suman sus campos y se compara el resultado con el máximo anterior. Se conservan tanto el contenido como el número de la mejor fila. La comparación estricta `sum > max_sum` hace que, si existe un empate, se mantenga la primera fila que alcanzó el máximo.

```
awk '{
    sum = 0
    for (i = 1; i <= NF; i++) sum += $i
    if (NR == 1 || sum > max_sum) {
        max_sum = sum
        best_row = $0
        best_idx = NR
    }
}
END {
    print "Fila:", best_idx, "(Suma =", max_sum ")"
    print best_row
}' matrix.txt
```

En conjunto, las soluciones usan procesamiento secuencial por flujo: salvo los arreglos de agregación requeridos en a), c) y e), no es necesario cargar los archivos completos en memoria. Esta propiedad hace que los comandos sean adecuados para datos mayores que la memoria principal, aunque las etapas globales de ordenamiento de b) pueden requerir almacenamiento temporal administrado por `sort`.

2 Parte 2: Alineamiento probabilístico mediante k -mers

El alineamiento exacto de lecturas cortas provenientes de secuenciación masiva (NGS) sobre referencias genómicas de gran escala supone una carga computacional prohibitiva mediante programación dinámica clásica (Smith-Waterman o Needleman-Wunsch). Como alternativa de filtrado previo, este proyecto implementa un método heurístico y probabilístico basado en semillas de largo fijo (k -meros) y conteo de votos de inicio relativo.

2.1 Definición del Problema y Concepto de *Hit*

El objetivo es predecir la coordenada P_{ref} más plausible donde comienza una lectura R de longitud $|R|$ dentro de una secuencia de referencia G . A diferencia de un alineador estándar, el sistema opera bajo las siguientes premisas:

- Un resultado **hit** indica exclusivamente que al menos una semilla compartida entre la lectura y la referencia sobrevivió a los filtros de colisión y propuso un origen consistente. No garantiza alineamiento de extremo a extremo ni ausencia de mutaciones.
- Un resultado **no-hit** ocurre cuando ninguna semilla de la lectura encuentra coincidencia utilizable en el índice, reportando posición 0, puntaje 0 y referencia comodín `*`.

2.2 Función de Hash y Aritmética de Preservación

Para transformar un k -mer en un índice de tabla en el rango $[0, M - 1]$, se utiliza una función basada en el algoritmo FNV-1a de 32 bits. Cada nucleótido se codifica como $v(A) = 0, v(C) = 1, v(G) = 2, v(T) = 3$. La función itera sobre cada base b_j de la semilla:

$$h_0 = 2166136261, \quad h_{j+1} = (h_j \times 16777619 + v(b_j)) \bmod 2^{32} \quad (1)$$

El índice final corresponde a:

$$\text{idx} = h_k \bmod M \quad (2)$$

Las letras minúsculas se homologan automáticamente a sus mayúsculas correspondientes. Cualquier carácter ambiguo (como nucleótidos desconocidos $\mathbf{N}$) invalida la semilla completa, provocando que se descarte del muestreo sin aportar votos.

2.3 Estructura de la Tabla y Descarte de Celdas

Cada ranura de la tabla hash almacena un contador de visitas y una coordenada:

```
typedef struct {
    uint32_t count;
    uint64_t position; // 1-based en el contig
} index_cell_t;
```

Al ingresar un k -mero en el índice, si la celda asociada posee $\text{count} = 0$, se registra la posición observada p_{ref} y se inicializa el contador en 1. Visitas subsiguientes únicamente incrementan el campo count . No existe resolución de colisiones (encadenamiento ni sondeo abierto).

El parámetro de filtrado c define el umbral de multiplicidad máxima permitida: durante la fase de votación, solo aquellas celdas con $1 \leq \text{count} \leq c$ son válidas para proponer candidatos. Aquellas con $\text{count} > c$ son descartadas por presumirse repetitivas o colisionadas.

2.4 Esquema de Votación y Resolución de Empates

Dada una lectura FASTQ, se extraen semillas con paso de muestreo L en desplazamientos relativos $p_{\text{read}} = j + 1$ (1-based). Para cada celda utilizable encontrada en la tabla, se propone la coordenada de inicio:

$$P_{\text{cand}} = P_{\text{ref}} - P_{\text{read}} + 1 \quad (3)$$

Se valida que $P_{\text{cand}} \geq 1$. Cada voto emitido se acumula en un vector de candidatos. La posición definitiva elegida corresponde a aquella con mayor puntaje (votos). En caso de empate estricto en el puntaje máximo, la ambigüedad se resuelve determinísticamente favoreciendo la coordenada numérica menor.

3 Ejemplo Manual Trazable (Toy Example)

Para transparentar los efectos intrínsecos de las colisiones y las reglas de resolución, se formula un caso mínimo evaluable a mano con $k = 3, L = 1, M = 8, c = 1$.

3.1 Configuración de Referencias e Inserción

Consideremos dos contigs de referencia en el FASTA:

- **Ref 1:** ACGT (Largo 4, semillas: ACG en pos 1, CGT en pos 2).
- **Ref 2:** TGCA (Largo 4, semillas: TGC en pos 1, GCA en pos 2).

Secuencia	Semilla ($k = 3$)	Posición (p_{ref})	Hash ($\text{idx mod } 8$)	Estado Final Celda
Ref 1	ACG	1	7	Celda 7: count=3 (Descartada, $> c$)
Ref 1	CGT	2	2	Celda 2: count=1, pos=2 (Válida)
Ref 2	TGC	1	7	Colisión en celda 7
Ref 2	GCA	2	7	Colisión en celda 7

Cuadro 1: Trazabilidad de inserciones en el índice de juguete ($M = 8, c = 1$).

3.2 Análisis de Lecturas de Prueba

1. **Lectura ACGT:** Semilla 1 (ACG) cae en celda 7 (descartada). Semilla 2 (CGT, $p_{\text{read}} = 2$) consulta celda 2 ($p_{\text{ref}} = 2$). Propone $P_{\text{cand}} = 2 - 2 + 1 = 1$. Resultado: **hit**, Ref 1, Posición 1, Score 1.
2. **Lectura TGCA:** Ambas semillas (TGC y GCA) caen en la celda 7. Al estar descartada por colisión, la lectura produce cero votos. Resultado: **no-hit** (Falso negativo causado por la falta de resolución de colisiones).
3. **Lectura GGG:** Su hash hipotéticamente coincide con la celda 2. Se genera un voto espurio para Ref 1 en pos 2. Resultado: **hit**, Score 1 (Falso positivo por colisión de hash no verificada).

4 Diseño de Software y Concurrency

4.1 Implementación en AWK

El script `aligner.awk` procesa el FASTA (`ARGIND==1`) almacenando la secuencia continua para evitar que el particionamiento de líneas corrompa los k -meros fronterizos. El índice se estructura sobre arreglos asociativos en memoria. Durante el FASTQ (`ARGIND==2`), las lecturas se procesan secuencialmente.

Causa de Discrepancia Numérica: En AWK estándar, los números enteros que superan $2^{31} - 1$ se convierten en representaciones de punto flotante de doble precisión (`double`), introduciendo pérdidas de precisión en la aritmética modular de FNV-1a frente a la aritmética modular nativa no signada de 64 bits de C (`uint64_t`). Esto explica por qué AWK registró un número radicalmente menor de celdas ocupadas y discrepancias en `diff`.

4.2 Implementación en C Secuencial

El programa `aligner_seq.c` reserva la tabla de M posiciones contiguas mediante un único llamado a `calloc`. La referencia se lee dinámicamente y se libera inmediatamente tras construir el índice para mantener bajo control la memoria residente (RSS). Las votaciones de cada lectura se gestionan en un arreglo denso estático local, resolviendo los ordenamientos en $\mathcal{O}(V^2)$ con $V \leq 1024$, lo que resulta altamente eficiente al operar íntegramente en la caché L1 del procesador.

4.3 Implementación en C Multihilo (Pthreads)

El modelo de paralelización adoptado en `aligner_pthreads.c` corresponde a un esquema **Productor-Consumidor multi-hilo con Reordering Buffer**:

- **Inmutabilidad del Índice:** La tabla hash se crea en la fase secuencial inicial. Al comenzar los hilos trabajadores, el índice pasa a ser de solo lectura. Esto elimina completamente la necesidad de semáforos o cerrojos de lectura/escritura (`rwlocks`) durante la consulta, permitiendo acceso concurrente libre de contención.

- **Cola Bloqueante de Trabajo:** El hilo principal lee de `stdin`, segmenta el flujo en bloques de $X = 1000$ lecturas y los introduce en una cola circular de tamaño acotado $Y = 64$. El acceso está sincronizado mediante un `pthread_mutex_t` y dos variables de condición: `not_full` (para suspender al productor si la cola se llena) y `not_empty` (para despertar a los consumidores). Todo despertar se ejecuta dentro de un bucle `while` para neutralizar despertares espurios (*spurious wakeups*).
- **Reordenamiento Determinista (FIFO):** Dado que hilos con distinta carga de cómputo pueden finalizar bloques fuera de orden, cada bloque lleva un `block_id` monotonico. Un hilo escritor dedicado (*Printer Thread*) aguarda en un arreglo de punteros completados la llegada del bloque en turno estricto (`next_to_print`). Esto garantiza que la salida estándar sea determinista e idéntica a la versión secuencial.

5 Evaluación Experimental y Resultados

5.1 Plataforma de Pruebas

Las pruebas experimentales se desarrollaron bajo el subsistema WSL2 sobre Linux Ubuntu 24.04 LTS (kernel 5.15.167.4-microsoft-standard-WSL2). El procesador anfitrión es un **Intel Core i5-10300H** con 4 núcleos físicos y 8 subprocesos lógicos a una frecuencia base de 2.50 GHz (Turbo hasta 4.50 GHz) y 8 MB de Intel Smart Cache. La memoria RAM total asignada al entorno fue de 7.7 GiB con 2.0 GiB de swap. Los binarios de C se compilaron con GCC 13.2.0 bajo el nivel de optimización `-O3`.

5.2 Resultados sobre *Escherichia coli*

Se utilizó el genoma de referencia `ecoli-k12-ref.fna` ($G = 4.641.652$ pb) y las lecturas `EC.50X.R1.fastq.gz` muestreadas a $N = 100.000$ lecturas (400.000 líneas). Los parámetros experimentales fijados fueron $k = 15, L = 10, M = 4.641.652, c = 1$. La tabla hash registró **439.404 celdas ocupadas**, lo que representa un factor de ocupación:

$$\alpha_{\text{ecoli}} = \frac{439.404}{4.641.652} \approx 0,0947 \quad (9,47\%) \quad (4)$$

Programa	Hilos	Tiempo (s)	CPU	Speedup	Eficiencia	RAM Pico
AWK Sec.	1	25.12	101%	0.05×	—	31.488 KB
C Sec.	1	1.27	93%	1.00×	100%	78.592 KB
C Pthreads	1	0.91	123%	1.39×	139%*	188.400 KB
C Pthreads	2	1.10	108%	1.15×	57.5%	188.400 KB
C Pthreads	4	0.82	117%	1.55×	38.7%	188.532 KB
C Pthreads	8	0.81	134%	1.57×	19.6%	188.404 KB

Cuadro 2: Métricas de ejecución en *E. coli* ($N = 100.000$ lecturas). *Nota: En $p = 1$ existen 2 hilos activos (consumidor y escritor), desacoplando la E/S.

Verificación de Concordancia: La ejecución de comparaciones binarias directas (`cmp -s`) certificó que *todas las salidas de C Pthreads (1, 2, 4 y 8 hilos) son exactamente idénticas a C Secuencial (diff: False)*.

Tasa de Procesamiento (Throughput): C Secuencial alcanzó una tasa de procesamiento de ≈ 78.740 lecturas/s, mientras que C Pthreads con 8 hilos elevó el rendimiento a ≈ 123.456 lecturas/s.

Distribución de Puntajes:

- Total de lecturas evaluadas: 100.000.
- Aciertos válidos (*hits*): 94.083 (94,08%).
- Sin alineamiento (*no-hits*): 5.917 (5,92%).
- Distribución: 88.352 lecturas (88,35%) obtuvieron puntaje 1. El resto se distribuyó entre puntajes de 2 a 29, con concentraciones notables en scores de 23 a 28 ($\approx 4,0\%$), correspondientes a lecturas sin errores que acumularon votos a lo largo de toda su extensión.

5.3 Resultados sobre Genoma Humano (*Homo sapiens*)

Para el genoma humano, las limitaciones de memoria contigua en entornos de 8 GB impiden dimensionar $M = G \approx 3.1 \times 10^9$ (requeriría ~ 49.6 GB de RAM). Se utilizó un contig acotado de pruebas (`GRCh38_test_small.fna`) y una muestra reproducible de 1.000 lecturas FASTQ generada mediante Python (`sample_1000.fq`) para evaluar la saturación de la tabla bajo $k = 31$, $L = 10$, $M = 1.000.000$, $c = 1$.

El índice humano registró *965.098 celdas ocupadas*, alcanzando un factor de carga crítico:

$$\alpha_{\text{humano}} = \frac{965.098}{1.000.000} \approx 0.9651 \quad (96.51\%) \quad (5)$$

Implementación	Hilos	Tiempo Real (s)	RAM Pico (KB)	Hits (%)	No-Hits (%)
C Secuencial	1	1.04	51.456	756 (75,60%)	244 (24,40%)
C Pthreads	4	1.09	51.328	756 (75,60%)	244 (24,40%)

Cuadro 3: Métricas sobre la muestra humana controlada ($N = 1.000$ lecturas).

Análisis de Calidad en Saturación: Las salidas fueron 100% idénticas (`diff: False`). No obstante, la tasa de *hits* se degradó a un 75,60% (con el 100% de ellos teniendo score unitario). Al estar ocupada el 96,51% de la tabla, una cantidad masiva de celdas acumuló múltiples visitas (`count > 1`), quedando completamente excluidas para la votación bajo la restricción $c = 1$.

6 Discusión y Análisis Teórico

6.1 Análisis Probabilístico de Colisiones

Bajo un modelo ideal donde la función de dispersión distribuye los k -meros de manera uniforme e independiente, la probabilidad de que dos semillas distintas colisionen en una misma celda es $p = \frac{1}{M}$. Si se indexan n semillas únicas, el valor esperado de pares de colisión $\mathbb{E}[C]$ responde a la paradoja del cumpleaños:

$$\mathbb{E}[C] \approx \frac{n(n-1)}{2M} \quad (6)$$

En el caso de *E. coli*, con $n \approx 464.165$ semillas indexadas y $M = 4.641.652$:

$$\mathbb{E}[C]_{\text{ecoli}} \approx \frac{(464.165)^2}{2 \times 4.641.652} \approx 23.208 \text{ colisiones} \quad (7)$$

Esto concuerda con la baja degradación observada ($\alpha \approx 9,5\%$). En contraste, para la prueba humana con $M = 10^6$ y $n \approx 1,5 \times 10^6$:

$$\mathbb{E}[C]_{\text{humano}} \approx \frac{(1,5 \times 10^6)^2}{2 \times 10^6} \approx 1.125.000 \text{ colisiones} \quad (8)$$

La tasa de colisión sobrepasó la capacidad de la tabla, forzando la ocupación al 96,51% y descartando masivamente k -meros válidos.

6.2 Cuellos de Botella y Ley de Amdahl

El escalamiento observado en Pthreads alcanzó un factor de aceleración de $1,57\times$ con 8 hilos sobre C secuencial, mostrando saturación a partir de 4 hilos. De acuerdo con la Ley de Amdahl:

$$S(p) = \frac{1}{(1 - f) + \frac{f}{p}} \quad (9)$$

donde f es la fracción paralelizable. En nuestra arquitectura, la carga global se compone de:

1. Descompresión al vuelo (`gzip -dc`) por una tubería (*pipe*) unidireccional de Linux (Secuencial).
2. Lectura e indexación FASTA inicial (Secuencial).
3. Consulta y cálculo de candidatos en paralelo (Paralelizable, f).
4. Consumo ordenado e impresión formateada por el *Printer Thread* hacia disco (Secuencial).

Dado que el tiempo secuencial basal de C ya es extremadamente reducido (1,27 segundos para 100.000 lecturas), la fracción serial ($1 - f$) domina rápidamente el tiempo de pared, tornando marginal el beneficio de añadir núcleos de procesamiento adicionales para cargas de este tamaño.

7 Conclusiones

- **Procesamiento con herramientas GNU/Linux:** La combinación de tuberías, AWK y `sort` permitió resolver los diez incisos de la primera parte sin generar copias descomprimidas del CSV. Los arreglos asociativos simplificaron las agregaciones por país y por columna, mientras que la lectura doble de `matrix.txt` permitió comparar promedios de fila con el promedio global.
- **Equivalencia Funcional y Concurrencia:** Se demostró reproducibilidad matemática estricta entre C Secuencial y C Pthreads en todas las configuraciones de hilos mediante un diseño de búfer de reordenamiento monotónico indexado por bloque.
- **Ineficiencia de Lenguajes Dinámicos:** AWK demostró ser inadecuado para bioinformática a escala de genoma completo: tardó 20 veces más tiempo que C, carece de soporte nativo para hilos y sus limitaciones en tipos de datos introducen fallas de precisión numérica en hashes modulares.
- **Gestión de Memoria en Escala Humana:** Para procesar el genoma humano completo en ordenadores de recursos estándar, es estrictamente mandatorio migrar hacia la indexación en *streaming* y codificación compacta de nucleótidos a 2 bits, previniendo los colapsos de memoria por asignación masiva contigua.